

Context

CLAS 生活服务平台当前定位为类似美团的校园/社区本地生活服务系统，技术栈为 Spring Boot、MySQL、Redis、Vue3、MyBatis Plus。项目已经从最小下单闭环扩展为多角色、多模块系统，包含用户端、商家端和管理员端，并已积累测试报告、架构说明、部署协作文档和数据库脚本。

课程目标不是单纯实现更多功能，而是提交一个规范完整的软件工程开发示例。因此评估标准应同时覆盖功能完整性、工程规范、文档一致性、测试可验证性和演示稳定性。

1. 已实现模块

角色/领域	已实现模块	当前成熟度
用户账号	手机号注册、登录、验证码、忘记密码、账号启用/禁用	中
权限控制	<code>Authorization</code> : <code><phone></code> 演示鉴权、 <code>@RequireRole</code> 角色控制、 <code>UserContext</code>	中低
商家浏览	商家列表、详情、搜索、分类、排序、地图/距离字段	中高
商品交易	商品列表、购物车、创建订单、订单明细、库存与金额	中高
支付履约	模拟支付、订单状态流转、配送状态、确认完成	中
评价体系	用户评价、商家回复、举报、管理员处理、评分重算	中高
地址管理	收货地址、默认地址、经纬度字段	中
收藏通知	收藏店铺、站内通知、关键业务通知联动	中
团购券	商家发布、用户购买、券码核销、券状态	中
预约服务	用户预约、取消、商家确认/完成/取消	中
商家端	入驻申请、审核状态、商品管理、订单处理、团购/预约管理	中高
管理后台	数据看板、用户管理、订单管理、商家审核、评价治理、公告管理	中高
数据库	16 张业务表、演示数据、重建脚本、非破坏性迁移脚本、H2 测试 schema	中
测试文档	功能测试报告、管理员测试报告、商品测试报告	中
部署协作	本地开发、服务器联调、数据库重建/迁移说明	中

核心业务闭环已经具备：

用户浏览商家 -> 加购物车 -> 创建订单 -> 模拟支付
-> 商家接单/配送 -> 用户确认完成 -> 用户评价

扩展闭环也已具备：

商家入驻 -> 管理员审核 -> 商家开店 -> 商品/团购/预约运营
评价举报 -> 管理员治理
用户退款 -> 商家审核 -> 通知用户

2. 缺失模块

缺失项	说明	对优秀课程设计的影响
需求规格说明书	缺少正式 SRS：业务目标、角色、用例、非功能需求	高
软件详细设计说明书正式版	现有架构说明偏摘要，缺少模块级设计、接口设计、流程图、状态机、异常设计	高
用户手册	缺少面向 USER/MERCHANT/ADMIN 的操作手册	高
部署文档正式版	<code>note.md</code> 很有价值，但需要整理为正式部署文档	高
文档口径统一	测试报告中账号、Auth Header、密码、分支、JDK 版本存在历史差异	高
JWT/Spring Security	当前鉴权适合演示，不适合正式系统	中
密码加密	当前演示数据和部分说明存在明文密码风险	中
统一错误码	未登录、无权限、业务错误多为 <code>code=400</code>	中
数据库约束	多数表缺外键、索引、唯一约束、状态约束说明	中
统一审计字段	表中 <code>created_at</code> 、 <code>updated_at</code> 、 <code>create_time</code> 命名不统一	中
完整迁移机制	只有一次迁移脚本，缺版本化迁移规范	中
E2E 演示脚本	缺少老师验收用的固定演示流程和测试数据复位方案	中
性能/安全测试	缺少并发、边界、安全风险测试记录	低到中

3. 功能优先级

优先级	工作项	理由
P0	统一文档口径与演示账号	直接影响提交可信度和答辩稳定性
P0	整理测试报告、部署文档、用户手册、详细设计说明书	课程目标要求，收益最高
P0	修正 README/测试报告中的历史账号、旧接口和旧状态描述	避免老师发现文档与系统不一致
P1	梳理订单、支付、退款、配送、评价、商家审核状态机	最容易被答辩追问
P1	完善数据库说明：表关系、字段含义、索引建议、金额单位、状态枚举	体现数据库设计能力
P1	前端补齐空状态、错误态、加载态、权限跳转提示	提升演示体验
P1	后端统一错误码与异常语义	提升工程规范
P2	密码 BCrypt、JWT 或 Spring Security	可作为安全增强亮点
P2	版本化 migration 或 Flyway/Liquibase	可作为工程化增强亮点
P2	E2E 自动化或 Postman/Apifox 集合	可作为测试增强亮点
P3	个性化推荐、优惠券营销、AI 客服、真实支付/短信	超出课程主线，只有在收口完成后再做

4. 用户体验问题

问题	影响	建议
多角色入口分散	用户、商家、管理员路径较多，答辩演示容易迷路	首页/登录后按角色进入固定工作台，文档给出演示路线
状态反馈不足	下单、支付、退款、预约、核销等流程如果失败，用户可能不知道下一步	每个关键操作提供成功/失败提示和下一步按钮
空状态不系统	无订单、无商品、无预约、无通知时可能显得页面空	统一空状态组件和引导动作
加载态不系统	接口慢时页面可能闪烁或无反馈	表格、列表、按钮增加 loading
表单校验不统一	商家入驻、商品、预约、地址、退款理由等表单规则分散	统一前端校验文案，和后端校验保持一致
移动端适配不确定	生活服务平台天然偏移动端，桌面可用不代表演示完整	至少保证 390px/768px/桌面三档核心页面可用
管理后台信息密度不一致	部分页面偏展示，部分页面偏操作	管理端保持表格、筛选、状态标签、操作按钮一致
地图能力展示弱	已有高德相关服务与组件，但课程答辩中需要可见亮点	在商家详情/下单页明确展示距离、预计配送时间或路线

5. 数据库设计问题

问题	影响	建议
缺少外键约束	数据关系依赖业务代码，容易产生孤儿记录	课程文档中至少画 ER 图并说明逻辑外键；实现可按风险逐步加物理外键
索引不足	商家列表、订单列表、用户消息、商家订单查询可能变慢	增加常用查询索引建议： <code>orders(user_id, create_time)</code> , <code>orders(merchant_id, status)</code> , <code>product(merchant_id, status)</code> , <code>notification(user_id, read_flag)</code>
状态字段散落为 VARCHAR	状态值靠代码白名单维护，数据库层没有约束	文档列出状态枚举；后续可使用 CHECK 或字典表
时间字段命名不统一	<code>created_at</code> 、 <code>updated_at</code> 、 <code>create_time</code> 混用	新增表统一 <code>created_at/updated_at</code> , 旧表文档说明历史原因
金额字段只有 INT 但说明不总是一致	前后端可能误把分当元	所有文档明确“金额单位为分，展示为元”
user 主键使用 phone	简化演示，但手机号变更困难，且与真实用户 ID 设计不同	课程中说明这是演示取舍；增强版可增加 <code>id</code> 主键并保留 <code>phone</code> 唯一
merchant.user_id 与多商家关系	当前 schema 使用 <code>UNIQUE KEY uk_merchant_user_id</code> , 限制一个账号一个商家	若课程场景为小平台可接受；若支持连锁店需改为一对多
migration 与 schema 可能漂移	旧库升级和新库初始化容易不一致	建立变更清单，所有字段变更同时更新 schema、migration、H2 schema、文档
缺少软删除统一策略	product 有 <code>DELETED</code> , 其他表未统一	明确哪些业务保留历史，哪些允许物理删除

6. 前后端架构问题

类型	问题	建议
后端鉴权	Authorization 直接传手机号，缺 token 过期、签名、防伪造	保持演示可用，同时在文档中标注限制；P2 增强 JWT/Spring Security
后端错误码	Result.fail 固定 400，未登录/无权限不能区分	扩展错误码：400/401/403/404/409/500 或业务码
后端分层	Service 已较完整，但部分业务规则和状态流转需要集中说明	将订单、退款、商家审核、预约状态机形成文档和测试
后端事务	下单扣库存、创建订单、支付流水等应保证事务一致性	检查关键 Service 是否使用事务，必要处补充 @Transactional
后端 DTO	DTO 较丰富，但响应对象和实体暴露边界需要继续统一	对外响应优先使用 Response DTO，避免泄露内部字段
前端路由	路由清晰，但 merchant-register 无明确角色 meta，公共/登录态边界需说明	明确每条路由 public/role/redirect 规则
前端 API	api/*.js 拆分清楚，但错误处理和 loading 可能分散	在 axios client 层统一错误提示和登录失效处理
前端状态	用户态基于 localStorage 与工具函数，适合演示但不够严谨	课程文档说明；增强版可用 Pinia 管理 auth/session
前端组件	有地图、公告、订单详情等组件，但表格/空状态/表单可复用不足	提炼基础组件提升一致性
测试架构	有 Maven 测试和文档测试，但自动化覆盖偏弱	增加核心 Service 集成测试和前端构建检查记录

7. 推荐的迭代开发路线

- 阶段 0：冻结范围与统一口径
- > 阶段 1：交付文档补齐
 - > 阶段 2：演示闭环打磨
 - > 阶段 3：工程质量修补
 - > 阶段 4：优秀亮点增强

阶段	目标	主要任务	交付物
阶段 0	冻结功能范围	确定最终演示账号、数据库脚本、演示流程、角色边界	统一口径清单
阶段 1	文档达标	编写/整理详细设计说明书、测试报告、部署文档、用户手册、需求说明	课程提交文档包
阶段 2	演示稳定	固定下单、商家处理、管理员治理、团购/预约演示路径；补空状态和提示	演示脚本和截图
阶段 3	工程规范	统一错误码、状态机说明、数据库索引建议、关键事务和校验	工程质量修复记录
阶段 4	亮点增强	选择 1-2 个亮点：地图配送、数据看板、JWT、迁移工具、E2E 测试	答辩亮点页

Goals / Non-Goals

Goals:

- Provide a truthful project assessment grounded in current code and documents.
- Prioritize work for achieving an excellent software engineering course submission.
- Convert broad issues into a staged roadmap with clear deliverables.

- Preserve the existing technology stack and avoid unnecessary late-stage feature expansion.
- Make the assessment usable as source material for final documentation and presentation.

Non-Goals:

- Implement code changes in this planning change.
- Replace the existing application architecture.
- Add large new business modules before documentation and demonstration quality are stable.
- Require production-grade security integrations before the course submission unless time permits.

Decisions

Decision 1: Treat documentation alignment as P0

The project already has many modules, so the biggest course-delivery risk is not missing business breadth but inconsistent engineering artifacts. Documentation alignment is prioritized over adding more features.

Alternative considered: add recommendation, coupon, or AI customer service modules first. This was rejected because these features increase testing and documentation burden without fixing submission credibility.

Decision 2: Keep authentication enhancement as P2

The current `Authorization: <phone>` flow is acceptable for a course demo if clearly documented as a simplified authentication mechanism. JWT/Spring Security is valuable but should not block final documentation and demonstration.

Alternative considered: immediately migrate to Spring Security. This was rejected because it risks destabilizing many role-protected APIs late in the project.

Decision 3: Focus database work on explanation, indexes, and consistency before heavy refactoring

The schema can support the course demo. The immediate need is to explain relationships, indexes, status fields, money units, and migration rules. Heavy primary-key or foreign-key refactoring should be deferred unless a specific bug blocks demonstration.

Alternative considered: redesign `user` with numeric ID and add all physical foreign keys. This is cleaner long-term but too risky near submission.

Decision 4: Use staged iteration instead of one large final sprint

The roadmap separates scope freeze, documentation, demo stabilization, engineering repair, and optional highlights. This makes progress visible and easier to divide among team members.

Alternative considered: split only by technical layers. This was rejected because the course grading evaluates full-process deliverables, not just backend/frontend completion.

Risks / Trade-offs

- [Risk] More documentation work may feel slower than coding new features -> Mitigation: reuse existing README, architecture, progress, schema, and test reports as source material.
- [Risk] Historical test reports contain old account and API conventions -> Mitigation: create a final consolidated test report and mark older reports as phase records.

- [Risk] Authentication remains demo-level -> Mitigation: document it honestly and list JWT/Spring Security as a known limitation or optional enhancement.
- [Risk] Database schema has limited constraints -> Mitigation: document logical relationships and add low-risk indexes/constraints only after verifying impact.
- [Risk] The team may over-expand features -> Mitigation: freeze business scope after current modules and focus on stable demonstration paths.
- [Risk] Final demo data may drift -> Mitigation: maintain one canonical `schema.sql` seed data set and one demo script.

Migration Plan

This planning change does not require runtime migration.

For later implementation:

1. Freeze current code branch and database seed data for final demo.
2. Update documentation sources from current code, routes, schema, and test reports.
3. Run backend tests, frontend build, and one manual full-process demo.
4. Apply only low-risk fixes before final packaging.
5. Keep `database/schema.sql` , `database/migration-20260608.sql` , and `backend/src/test/resources/schema-test.sql` synchronized for any schema adjustment.

Rollback strategy:

- Planning artifacts can be archived or removed without affecting runtime code.
- Later implementation tasks should be committed in small batches so documentation-only changes, UX fixes, and backend/database fixes can be reverted independently.

Open Questions

- Does the final course requirement specify exact document templates, page counts, or naming conventions?
- Will the teacher evaluate via live demo, source review, submitted video, or all three?
- Should the final system present itself as a campus-focused platform or a broader city/community life service platform?
- Is JWT/Spring Security required by the course rubric, or can simplified demo authentication be accepted with limitation notes?
- Should older phase test reports remain as appendices, or should the team submit only one consolidated final test report?